إعداد الطالبة: فاطمة الزهراء اسلام نجاح صادق محمد
الكود الجامعي: 2500146
معهد الدلتا العالي لنظم المعلومات الإدارية والمحاسبية

الذكاء الاصطناعي التوليدي كعامل مساعد للمطورين

المقدمة: المفهوم والنشأة

أصبحت البرمجيات جزءًا أساسيًا من الخدمات التعليمية والتجارية والإدارية، ولذلك يبحث المطورون عن وسائل تساعدهم على إنجاز العمل بدقة وسرعة مع الحفاظ على جودة المنتج النهائي. ومن التقنيات المهمة في هذا المجال الذكاء الاصطناعي التوليدي، وهو نوع من الذكاء الاصطناعي يستطيع إنشاء محتوى جديد مثل النصوص والأكواد والصور والملخصات اعتمادًا على الأنماط التي تعلمها من البيانات التي دُرّب عليها.

في مجال البرمجة، يمكن لهذه الأنظمة شرح وظيفة برمجية، أو اقتراح إكمال سطر من الكود، أو إنشاء مثال باستخدام HTML وCSS، أو المساعدة في فهم رسالة خطأ. وهي بذلك توفر نقطة بداية وتسهل تجربة الأفكار، لكنها لا تضمن أن كل إجابة صحيحة أو مناسبة للمشروع؛ فقد ينتج النموذج كودًا يبدو منطقيًا لكنه لا يعمل أو لا يراعي جميع متطلبات التطبيق.

ترجع جذور أبحاث الذكاء الاصطناعي إلى منتصف القرن العشرين. ومع تطور قدرات الحوسبة وتوافر البيانات، تقدمت أساليب التعلم الآلي. وفي العقد الثاني من القرن الحادي والعشرين، ساعد التعلم العميق والشبكات العصبية على تحسين معالجة اللغة والصور. ثم انتشرت النماذج اللغوية القادرة على توليد النصوص والأكواد، وأصبحت أدوات المحادثة والمساعدات البرمجية منذ عام 2022 أقرب إلى الطلاب والمطورين والشركات.

أهمية استخدامه في تطوير الويب

لا تقتصر فائدته على كتابة الشيفرة، بل يمكن استخدامه في مراحل متعددة: فهم الفكرة، وتحديد مكونات الصفحة، وإعداد نموذج أولي، وشرح الكود، واقتراح الاختبارات، وتشخيص الأخطاء، وتحسين قابلية الاستخدام. ويساعد ذلك على تسريع المهام المتكررة وتوسيع الخيارات المتاحة للمطور، مع بقاء فهم المشروع ومراجعته واتخاذ القرار النهائي مسؤولية الإنسان.

متى تكون فائدته أكبر؟

تكون النتائج عادة أفضل عندما تكون المهمة محددة، ويُذكر السياق والتقنيات المطلوبة والقيود وشكل النتيجة. فبدلًا من طلب «إنشاء موقع كامل»، يمكن طلب بطاقة تعريف صغيرة باستخدام HTML وCSS، ثم مراجعتها وتطويرها خطوة بخطوة. ويساعد تقسيم العمل إلى مهام قصيرة على اكتشاف الأخطاء مبكرًا وفهم كل جزء قبل دمجه في المشروع.

كيف يساعد الذكاء الاصطناعي التوليدي المطورين؟

أولًا: شرح الأكواد

يمكن إدخال مقطع برمجي وطلب شرح وظيفته خطوة بخطوة، أو توضيح معنى أمر أو دالة أو متغير أو رسالة خطأ. وتفيد هذه الميزة الطلاب والمبتدئين في فهم المفاهيم والتعرف إلى أكثر من طريقة لحل المشكلة. ويمكن أيضًا طلب شرح سبب استخدام جزء معين، أو مقارنة حلين من حيث الوضوح وسهولة الصيانة. ومع ذلك، يجب مقارنة الشرح بالكود الفعلي والرجوع إلى التوثيق الرسمي؛ فقد يسيء النموذج تفسير السياق أو يغفل حالة خاصة.

ثانيًا: إنشاء أمثلة برمجية

يمكن للأداة توليد نموذج أولي لصفحة ويب، أو نموذج إدخال، أو قائمة تنقل، أو تنسيق CSS اعتمادًا على وصف مكتوب. وتساعد النماذج الأولية على تجربة الفكرة بسرعة قبل استثمار وقت طويل في تنفيذها. ومن الأفضل تحديد لغة البرمجة والتقنيات المسموح بها، وطلب فصل HTML عن CSS، واستخدام أسماء واضحة للعناصر، وشرح وظيفة الأجزاء المهمة. بعد ذلك يراجع المطور الكود ويختبره في المتصفح ولا ينسخه مباشرة إلى مشروع فعلي دون فهم.

ثالثًا: اكتشاف الأخطاء

يمكن تقديم رسالة الخطأ والجزء المرتبط بها وطلب اقتراح الأسباب المحتملة وخطوات التشخيص. وقد تساعد الأداة في مراجعة المتغيرات، وترتيب العناصر، وقواعد CSS، أو اقتراح حالات اختبار. ولكي تكون المساعدة أدق، ينبغي توضيح ما كان متوقعًا وما حدث بالفعل، وذكر الخطوات التي تؤدي إلى المشكلة. ثم تُجرّب الإصلاحات واحدة تلو الأخرى، ويُعاد تشغيل الصفحة للتأكد من زوال الخطأ وعدم ظهور أخطاء جديدة.

رابعًا: التخطيط وتحسين الصفحة

يمكن وصف فكرة الموقع وطلب تقسيمها إلى صفحات ومكونات، مثل قائمة تنقل ونموذج تواصل وبطاقات محتوى. كما يمكن اقتراح تنظيم العناوين والفقرات والأزرار، وتنسيق الألوان والخطوط والمسافات، ومراعاة اتجاه الكتابة من اليمين إلى اليسار في الصفحات العربية. ويساعد التخطيط قبل كتابة الكود على توضيح المطلوب وتقليل التعديلات المتأخرة، لكن ينبغي التأكد من أن التصميم يخدم غرض الصفحة ولا يضيف عناصر غير ضرورية.

خطوات استخدام المساعدة بفاعلية

- تحديد المشكلة أو المهمة بدقة، وذكر اللغة والتقنيات المستخدمة.
- طلب حل صغير قابل للفهم بدلًا من مشروع ضخم دفعة واحدة.
- قراءة الاقتراح وطلب توضيح الأجزاء غير المفهومة قبل اعتماده.
- تشغيل الكود واختبار الوظائف الأساسية، ثم تحسينه تدريجيًا.

أشهر أدوات الذكاء الاصطناعي للمطورين

ChatGPT

يتيح ChatGPT إجراء محادثة لشرح المفاهيم البرمجية، واقتراح حلول، وكتابة أمثلة أولية، وتلخيص المعلومات، والمساعدة في فهم الأخطاء. ويمكن للمطور أن يطلب أكثر من صيغة للحل أو أن يطلب شرح الكود بلغة بسيطة. وتعتمد جودة النتيجة على وضوح التعليمات والسياق المقدم؛ لذلك من المفيد إرفاق الجزء الضروري من الكود ووصف المشكلة بدلًا من الاكتفاء بسؤال عام. ويجب فحص المخرجات والتحقق من صحتها قبل استخدامها.

GitHub Copilot

يعمل GitHub Copilot كمساعد برمجي داخل بيئات تطوير مدعومة، ويقدم اقتراحات لإكمال الكود أثناء الكتابة، وقد يوفر خصائص للشرح أو الاختبار وفق المنتج والإعدادات المتاحة. ومن مزاياه أن الاقتراح يظهر بالقرب من مكان العمل، فيستطيع المطور تجربته وتعديله. لكن الاقتراح ليس بديلًا عن فهم البرنامج؛ إذ يجب مراجعته واختباره والتحقق من توافقه مع متطلبات المشروع وسياسات الخصوصية والملكية الفكرية.

Google Gemini

تقدم Google نماذج Gemini وواجهات برمجية يمكن استخدامها لتوليد النصوص وتحليلها وبناء تطبيقات تستفيد من النماذج. وقد تشمل الاستخدامات إعداد نماذج أولية، ومعالجة المدخلات، وتوليد مخرجات منظمة بحسب إمكانات النموذج والواجهة. ويتعين على المطور الرجوع إلى الوثائق الرسمية لمعرفة طريقة التكامل وحدود الاستخدام والخصائص المتاحة، لأن هذه التفاصيل قد تتغير مع تحديث الخدمات.

اختيار الأداة المناسبة

تختلف الأدوات في طريقة العمل والتكامل والخصائص المتاحة، لذلك يعتمد الاختيار على طبيعة المهمة والبيئة البرمجية ومستوى خبرة المستخدم وسياسات الخصوصية. قد تكون أداة المحادثة مناسبة لفهم مفهوم أو مقارنة حلول، بينما يكون المساعد المدمج في بيئة التطوير مناسبًا لاقتراح إكمال الكود أثناء الكتابة. ومن الأفضل تجربة الأداة على مهمة صغيرة وتقييم دقة النتيجة ووضوحها وسهولة مراجعتها قبل الاعتماد عليها في عمل أكبر.

تطبيقات عملية في مشروع ويب

يمكن دمج الأدوات في مراحل المشروع: تنظيم الفكرة وتقسيمها إلى مكونات، إنشاء نموذج أولي، شرح أجزاء الكود، اقتراح اختبارات، ثم المساعدة في تشخيص الأخطاء. ويُفضّل أن تكون كل مهمة محددة ومستقلة قدر الإمكان، وأن يحتفظ المطور بالنسخ التي يعرف أنها تعمل حتى يستطيع الرجوع إليها عند حدوث مشكلة.

خطوات عملية لبناء صفحة ويب بمساعدة الذكاء الاصطناعي

يمكن توضيح طريقة العمل من خلال مثال صفحة تعريفية لمبادرة أو نشاط طلابي. الهدف هو الاستفادة من سرعة الأداة مع الحفاظ على الفهم والتحكم في النتيجة.

1. تحديد المطلوب قبل كتابة الكود

يبدأ العمل بوصف هدف الصفحة والجمهور المستهدف، ثم تحديد العناصر الأساسية مثل عنوان واضح، وفقرة تعريفية، وصورة مناسبة، وزر للتواصل. ويمكن تحديد قيود مثل استخدام HTML وCSS فقط، ودعم اللغة العربية، واتجاه الكتابة من اليمين إلى اليسار، وملاءمة التصميم لشاشات الهاتف والحاسوب. كلما كان الوصف واضحًا، اقتربت الاقتراحات من الحاجة الفعلية وقلّ احتمال الحصول على عناصر لا يحتاجها المشروع.

2. طلب نموذج أولي قابل للتعديل

بعد تحديد المتطلبات، يُطلب مثال صغير بدلًا من طلب موقع كامل دفعة واحدة. ومن المفيد مطالبة الأداة بفصل HTML عن CSS، واستخدام أسماء واضحة للعناصر، وشرح وظيفة كل جزء. ويمكن البدء بعنوان وفقرة وزرٍ واحد، ثم إضافة العناصر الأخرى بعد اختبار النموذج الأولي. بهذه الطريقة يستطيع الطالب فهم البنية وتعديلها بنفسه بدلًا من نسخ كود لا يعرف طريقة عمله.

3. الاختبار على أكثر من شاشة

لا يكفي التأكد من صحة الصفحة على شاشة الحاسوب؛ بل ينبغي فتحها على شاشة صغيرة أيضًا، والتأكد من وضوح النصوص وعدم تداخل العناصر، وتجربة الأزرار والروابط. وعند ظهور مشكلة، يُرسل وصفها والجزء المرتبط بها إلى المساعد مع توضيح المتوقع وما حدث بالفعل. وبعد التعديل، يعاد الاختبار للتأكد من أن الإصلاح حل المشكلة ولم يؤثر في أجزاء أخرى.

4. مراجعة الوصول والأمان

تتضمن المراجعة استخدام العناوين الدلالية، وتسميات واضحة لحقول النماذج، ونصوص بديلة للصور، وتباين مناسب بين النص والخلفية، وإمكانية التنقل باستخدام لوحة المفاتيح. كما ينبغي عدم إدخال كلمات المرور أو مفاتيح الخدمات أو بيانات الأشخاص في المحادثة. فالمراجعة البشرية مهمة لأن الاقتراح الآلي قد يغفل مشكلة لا تظهر من النظرة الأولى.

مثال على تعليمات واضحة للأداة

«أنشئ بطاقة تعريف عربية بسيطة باستخدام HTML وCSS فقط. اجعل اتجاه الصفحة من اليمين إلى اليسار، واستخدم عنوانًا وفقرة وزرًا واضحًا، واجعل التصميم مناسبًا للهاتف. افصل التنسيق في قسم مستقل، واشرح وظيفة العناصر، ثم اقترح قائمة قصيرة لاختبار الصفحة قبل استخدامها».

كيف نقيّم النتيجة؟

- هل تنفذ الصفحة الهدف المطلوب دون عناصر زائدة؟
- هل الكود واضح ويمكن للمطور شرحه وتعديله؟
- هل تعمل الروابط والأزرار وتظهر الصفحة بشكل مناسب على الهاتف؟
- هل تمت مراجعة الوصول والأمان والبيانات المستخدمة؟

التحديات والمخاطر ومبادئ الاستخدام المسؤول

أخطاء ومخرجات غير دقيقة

قد يقترح النموذج كودًا لا يعمل، أو يستخدم واجهة برمجية قديمة، أو يذكر مكتبة أو خاصية غير موجودة. وقد يبدو الناتج منطقيًا رغم وجود خلل. ولتقليل المخاطر، يجب تشغيل الكود وكتابة اختبارات مناسبة والرجوع إلى الوثائق الرسمية ومراجعة الحالات الحدية قبل الاعتماد عليه. كما ينبغي عدم اعتبار الثقة في صياغة الإجابة دليلًا على صحتها.

الأمن والخصوصية

قد يؤدي إدخال بيانات سرية أو شيفرات داخلية إلى معالجتها وفق إعدادات الخدمة وسياساتها. وقد ينتج النموذج أيضًا كودًا غير آمن، مثل التحقق الضعيف من المدخلات أو التعامل غير السليم مع الصلاحيات. لذلك يجب تجنب مشاركة كلمات المرور ومفاتيح API وبيانات العملاء والتفاصيل الداخلية، واستخدام بيانات اختبار، ومراجعة المصادقة والتحقق من المدخلات، وإجراء فحص أمني قبل إطلاق التطبيق.

التحيز والملاءمة

تعكس مخرجات النماذج جزئيًا الأنماط الموجودة في بيانات التدريب، وقد تختلف جودة الأمثلة باختلاف اللغة أو السياق. وفي تطوير الويب، قد يؤدي ذلك إلى نص غير مناسب أو تجربة لا تراعي جميع المستخدمين. لذا يجب تقييم المخرجات ضمن السياق الحقيقي، ومراجعة سهولة الاستخدام وإمكانية الوصول، وعدم افتراض أن النموذج محايد أو مناسب دائمًا.

مبادئ الاستخدام المسؤول

- تحديد المهمة بوضوح وكتابة تعليمات تتضمن السياق والقيود وشكل الناتج المطلوب.
- مراجعة المخرجات وفهم الكود وتشغيله والتحقق من السلوك المتوقع وعدم تعطيل وظائف موجودة.
- حماية الخصوصية وعدم إرسال أسرار أو بيانات شخصية دون تصريح وسياسة واضحة.
- احترام حقوق الملكية الفكرية والتراخيص والالتزام بمتطلبات الدراسة أو العمل، وتوثيق دور الأداة عند الحاجة.
- استخدام الذكاء الاصطناعي لدعم التعلم لا لتجاوز متطلباته، والتمييز بين ما كتبه المستخدم وما اقترحته الأداة وما تم التحقق منه.

الخلاصة

يمثل الذكاء الاصطناعي التوليدي فرصة مهمة لتطوير العمل البرمجي؛ فهو يساعد على شرح الأكواد، وإنشاء أمثلة أولية، واقتراح حلول للأخطاء، وتحسين بعض جوانب الوصول. وتتيح أدوات مثل ChatGPT وGitHub Copilot وGemini الاستفادة من هذه الإمكانات عبر المحادثة أو التكامل مع بيئة التطوير أو الواجهات البرمجية. لكن قيمة الأدوات لا تقاس بسرعة إنتاج النص أو الكود وحدها، بل بصحته وملاءمته ووضوحه. وتظل مراجعة المطور واختباره وحماية البيانات واحترام الحقوق عناصر أساسية لرفع الإنتاجية ودعم التعلم دون إلغاء مسؤولية المطور.

مراجع للاطلاع

OpenAI — Prompt Engineering؛ Google — Gemini API Documentation؛ MDN — Learn Web Development؛ GitHub Copilot Documentation؛ UNESCO — Guidance for generative AI in education and research؛ NIST — Generative AI Risk Management Profile